About Locking and cancelling/terminating queries

Link to the video of the runbook simulation.

A `lock` in PostgreSQL is a feature that allows transactions to *hold* something. Usually, that *something* is a table, an index, or a portion (row/s) of these.

Locks are the way PostgreSQL ensure transactional data consistency, and this is specially handy when concurrent transactions tries to write the same data.

PostgreSQL implements several kinds and layers of locks, and even SELECT statements implements a kind of lock. Locks will execute concurrently, except when one lock conflicts with another.

Below is a comparison of the most commonly used SQL commands that can run concurrently with each other on the same table:

Runs concurrently with	SELECT	INSERT UPDATE DELETE	VACUUM ANALYZE	CREATE INDEX (CONC)	CREATE INDEX	CREATE TRIGGER	ALTER TABLE DROP TABLE TRUNCATE VACUUM (FULL)
SELECT	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:
INSERT UPDATE DELETE	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:x:
CREATE INDEX (CONC) VACUUM ANALYZE	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:x:	:x:	:x:	:x:
CREATE INDEX	:heavy_check_mark:	:x:	:heavy_check_mark:	:x:	:x:	:heavy_check_mark:	:x:
CREATE TRIGGER	:heavy_check_mark:	:x:	:heavy_check_mark:	:x:	:x:	:x:	:x:
ALTER TABLE DROP TABLE TRUNCATE VACUUM (FULL)	:x:	:x:	:x:	:x:	:x:	:x:	:x:

And here is a list of locks each statement implements at table and row level:

`SELECT`: ACCESS SHARE LOCK

`INSERT UPDATE DELETE`: ROW EXCLUSIVE LOCK

`VACUUM`, `ANALYZE`, `CREATE INDEX (conc)`: SHARE UPDATE EXCLUSIVE LOCK

`CREATE INDEX`: SHARE LOCK

`CREATE TRIGGER`, `ALTER TABLE`: SHARE ROW EXCLUSIVE LOCK

`VACUUM FULL`, `REINDEX`, `TRUNCATE`: ACCESS EXCLUSIVE LOCK

More information in the Official docs

==========

How to see locking and locked activity

When troubleshooting blocked queries, two internal sources of information comes in handy:

The pg_stat_activity view, wich gives information about current activity, where is connected from, wich query is executing, and the

pg_locks system catalog, that provides information current locking activity, wich pid it belongs to, if the lock where granted (or is waiting), and so on.

For demostration porpouses, we will create an artificial lock using the LOCK command:

Session :one: will issue a LOCK statement (wich implement a ACCESS EXCLUSIVE lock, wich conflicts even with SELECT operations:

```
locktest=# begin;
BEGIN
locktest=# lock TABLE mytable ;
LOCK TABLE
```

Then session :two: try to access the same data:

```
locktest=# select count(*) from mytable;
```

That SELECT will wait until LOCK is released. OK, how can we tell what is blocking what? We can use this snippet posted by Nikolay Samokhvalov, that combines the information from `pg_stat_activity` and `pg_locks`:

```
with recursive l as (
  select
    pid, locktype, granted,
    array_position(array['AccessShare','RowShare','RowExclusive','ShareUpdateExclusive','Sha
    row(locktype, database, relation, page, tuple, virtualxid, transactionid, classid, objic
  from pg_locks
), pairs as (
  select w.pid waiter, l.pid locker, l.obj, l.m
  from l w join l on l.obj is not distinct from w.obj and l.locktype = w.locktype and not l.
  where not w.granted
  and not exists (select from l i where i.pid=l.pid and i.locktype = l.locktype and i.obj is
), leads as (
  select o.locker, 1::int lvl, count(*) q, array[locker] track, false as cycle
  from pairs o
```

```
  group by o.locker
  union all
  select i.locker, leads.lvl + 1, (select count(*) from pairs q where q.locker = i.locker),
  from pairs i, leads
  where i.waiter=leads.locker and not cycle
), tree as (
  select locker pid,locker dad,locker root,case when cycle then track end dl, null::record c
  from leads o
  where
    (cycle and not exists (select from leads i where i.locker=any(o.track) and (i.lvl>o.lvl
    or (not cycle and not exists (select from pairs where waiter=o.locker) and not exists (s
  union all
  select w.waiter pid,tree.pid,tree.root,case when w.waiter=any(tree.dl) then tree.dl end,w.
  from tree
  join pairs w on tree.pid=w.locker and not w.waiter = any (all_pids)
)
select (clock_timestamp() - a.xact_start)::interval(0) as transaction_age,
  (clock_timestamp() - a.state_change)::interval(0) as change_age,
  a.datname,
  a.usename,
  a.client_addr,
  tree.pid,
  a.wait_event_type,
  a.wait_event,
  pg_blocking_pids(tree.pid) blocked_by_pids,
  replace(a.state, 'idle in transaction', 'idletx') state,
  lvl,
  (select count(*) from tree p where p.path ~ ('^'||tree.path) and not p.path=tree.path) blo
  case when tree.pid=any(tree.dl) then '!>' else repeat(' .', lvl) end||' '||trim(left(regex
from tree
left join pairs w on w.waiter = tree.pid and w.locker = tree.dad
join pg_stat_activity a using (pid)
join pg_stat_activity r on r.pid=tree.root
order by (now() - r.xact_start), path
```

This will show a complete *tree* of blocking and blocked queries:

```
 transaction_age | change_age | datname  | usename  | client_addr |  pid  | wait_event_type
-----------------+------------+----------+----------+-------------+-------+----------------
 00:10:25        | 00:10:18   | locktest | postgres | 127.0.0.1   | 20252 | Client
 00:09:14        | 00:09:14   | locktest | postgres | 127.0.0.1   | 20594 | Lock
(2 rows)
```

Here, the *root* of the blocking chain can be identified with a 0 (zero) in the `lvl` column.

In practice, you will probably add a `\watch 2` in *gitlab-psql* session, for automatically repeat and refresh the screen.

How to cancel (or terminate) a blocking query

If you decided to cancel the blocking query, you only need to take that from the `pid` column and execute a

```
select pg_cancel_backend(<pid>);
```

or

```
select pg_terminate_backend(<pid>);
```

Both functions will return `true` or `false`, meaning that operation was successfully achieved (or not).

The difference between each of those, is that `pg_cancel_backend()` sends a SIGINT (cancels only the current query, but let's the connection continue to execute queries - idle connections or connections in transaction can't be canceled) and `pg_terminate_backend()` sends a SIGTERM signal (terminating the backend process immediately, also terminating the whole connection). Since terminating any process with SIGTERM can lead to undesired results, you should always try `pg_cancel_backend()` first.

Other options would entail restarting the local Patroni connection to the DB; this would help resetting Postgres TCP state if we believe Postgres is in trouble at the network layer.

A final option, in case pg_cancel_backend or pg_terminate_backend does not terminate the session, would be to execute a kill from the session that is generating the lock. However this could case instability in the Postgres main process so we could instead think of restarting the Postgres process entirely, for example if the node were out of rotation.

How to check if queries are waiting to aquire locks from the logs

If log_lock_waits is `on`, then every attempt to acquire a lock that has been waiting for more than deadlock_timeout, a line will be printed to the logfile, similar to:

```
2020-06-26 06:42:37.472 GMT,"gitlab","gitlabhq_production",63330,"10.217.8.4:44814",5ef59793
0.126 ms","Process holding the lock: 63387. Wait queue: 41595, 58665, 61792, 63327.",,,,"whi
6) in relation ""issues""","UPDATE ""issues"" SET ""updated_at"" = '2020-06-26 06:42:31.9532
.2.7 queues:pipeline_...ate_highest_role [1 of 5 busy]"
```

depending of the sql being executed.

Similary, locks that took more than to be acquired will also log a message:

```
2020-06-26 07:06:02.604 GMT,"gitlab","gitlabhq_production",92156,"10.217.4.2:49126",5ef59e29
```

That can be tracked down to see if there are queries that have been waiting for too long.

Deadlocks

A deadlock is a situation where two (or more) processes conflict on their use of resources, each one needing to lock to resource being already locked by the other one, hence blocking each other. PostgreSQL comes with an deadlock detection routine that will kill one of the process involved, allowing the other(s) to progress.